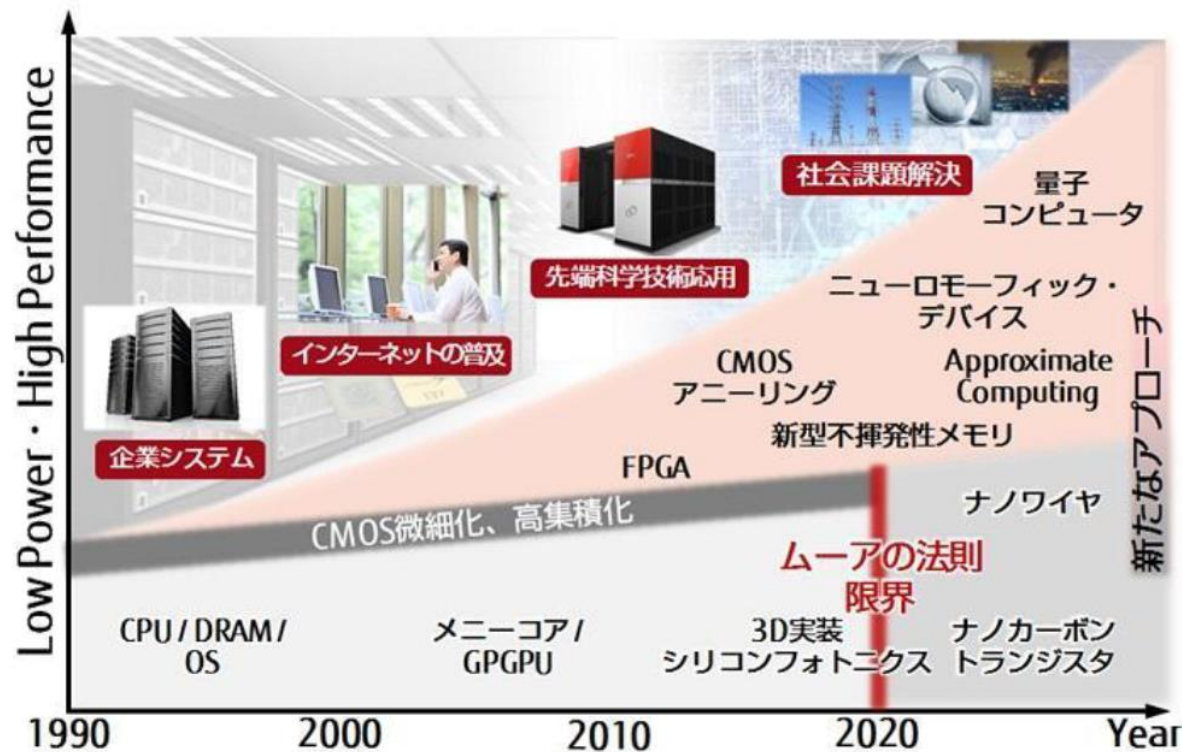


計算機アーキテクチャ

第8章 スーパスカラとVLIW

8.1 高速化手法



8.1 高速化手法

命令 1	Fetch	Decode	Execute	Cache	Write	
命令 2	Fetch	Decode	Execute	Cache	Write	



命令 1	Fetch	Decode	Execute	Cache	Write	
命令 2	Fetch	Decode	Execute	Cache	Write	

(a) 動作周波数だけ向上できれば効果は大, しかし, トランジスタの動作速度に依存

命令 1	Fetch	Fetch	Decode	Decode	Execute	Execute	Cache	Cache	Write	Write
命令 2	Fetch	Fetch	Decode	Decode	Execute	Execute	Cache	Cache	Write	Write

(b) 実際には難しいので, パイプライン段数を増やして見掛けの周波数を上げる

命令 1	F	F	F	F	D	D	D	D	E	E	E	E	C	C	C	C	W	W	W	W
命令 2	F	F	F	F	D	D	D	D	E	E	E	E	C	C	C	C	W	W	W	W

(c) どんどん増やすと, 複数命令の同時実行と同じ

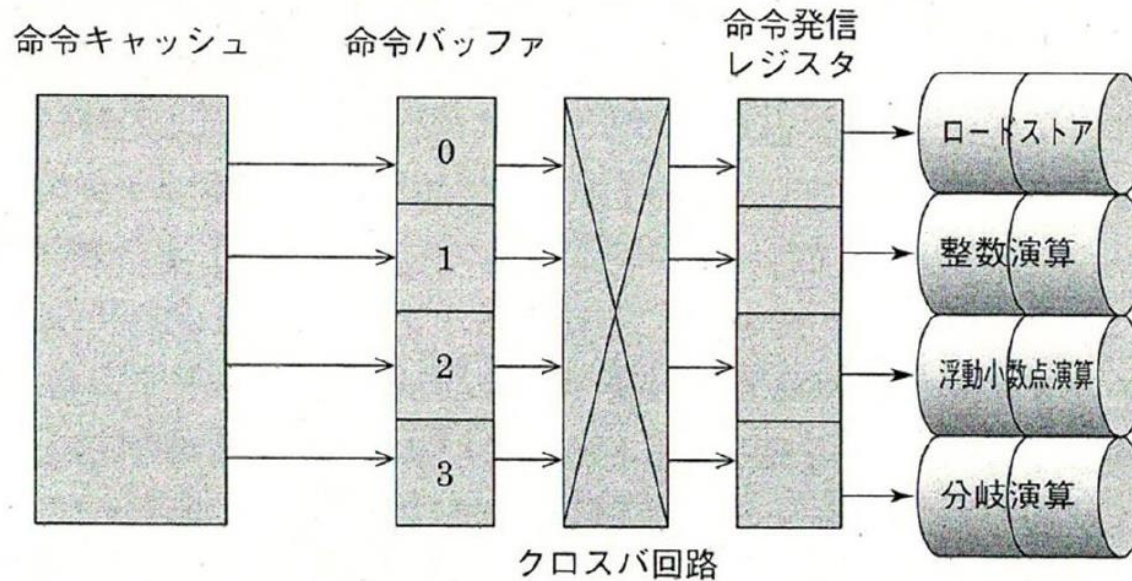
命令 1	F	F	F	F	D	D	D	D	E	E	E	E	C	C	C	C	W	W	W	W
命令 2	F	F	F	F	D	D	D	D					E	E	E	E	C	C	C	C

(d) 依存関係があると同時実行できないので元の木阿弥

命令 1	F	F	F	F	D	D	D	D	E	E	E	E	C	C	C	C	W	W	W	W
命令 2													F	F	F	F	D	D	D	D

(e) 分岐予測が外れた場合の損失も大きい ↑ 分岐予測ミスによる命令再フェッチ

8.2 スーパースカラとレジスタリネーミング



複数ステージが、それぞれ命令を担当する

プログラムの論理が崩れないよう、命令の順番を変えながらパイプラインにつなげていく

命令間の依存関係（単純なパイプライン含む）

パイプラインの待ち（ ）、パイプラインの乱れ（ ）
これらを起こす可能性のあるもの.

(a) フロー依存

(b) 逆依存

(c) 出力依存

命令のパイプライン化により問題になる.

命令間の依存関係

(a) フロー依存
回避困難

(b) 逆依存
レジスタリネーミングで回避

(c) 出力依存
レジスタリネーミングで回避

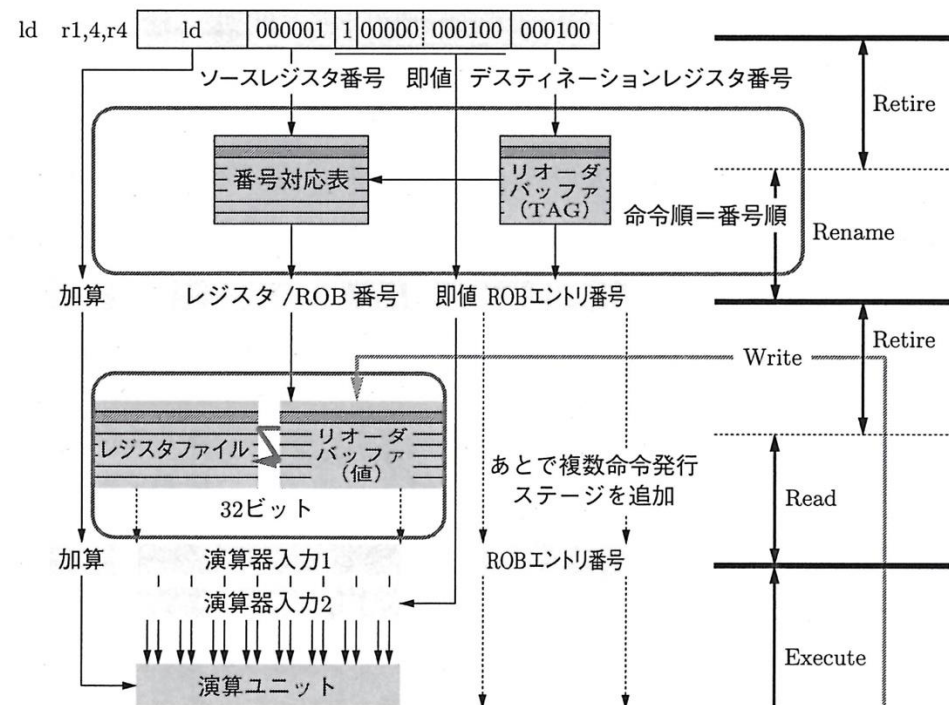
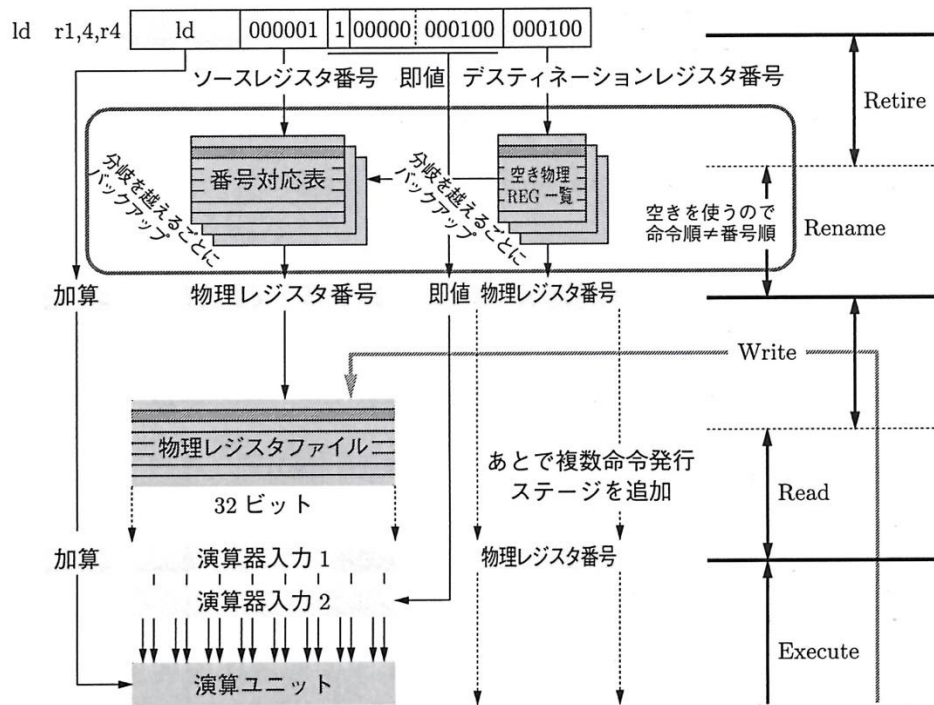
(a) フロー依存 (Write $\Rightarrow$ Read) 先行命令のデスティネーション = 後続命令のソース		
1. add r1,r2 $\rightarrow$ r8		
2. sub r8,r3 $\rightarrow$ r4	※ r8 は本質的データ依存	
(b) 逆依存 (Read $\Rightarrow$ Write) 先行命令のソース = 後続命令のデスティネーション		
1. sub r8,r3 $\rightarrow$ r4		
2. add r5,r6 $\rightarrow$ r8	※ 書込み先 r8 を変更して回避可能	
(c) 出力依存 (Write $\Rightarrow$ Write) 先行命令と後続命令のデスティネーションが一致		
1. add r1,r2 $\rightarrow$ r8		
2. sub r8,r3 $\rightarrow$ r4	※ r8 は本質的データ依存	
3. add r5,r6 $\rightarrow$ r8	※ 書込み先 r8 を変更して回避可能	
4. sub r8,r3 $\rightarrow$ r7	※ r8 は本質的データ依存	
(d) デスティネーションに一時レジスタを割当て		
1. add r1,r2 $\rightarrow$ p0(r8)	※ r8 に代えて p0	
2. sub r8,r3 $\rightarrow$ p1(r4)	※ r4 に代えて p1	
3. add r5,r6 $\rightarrow$ p2(r8)	※ r8 に代えて p2	
4. sub r8,r3 $\rightarrow$ p3(r7)	※ r7 に代えて p3	
(e) 対応するソースを変更して 2 命令ずつ同時実行可能		
1. add r1,r2 $\rightarrow$ p0		
2. sub p0,r3 $\rightarrow$ p1	※ p0 は本質的データ依存	
3. add r5,r6 $\rightarrow$ p2		
4. sub p2,r3 $\rightarrow$ p3	※ p2 は本質的データ依存	

8.3 スーパースカラの命令スケジューリング

[1] インオーダー実行とアウトオーダー実行



[2] 物理レジスタとリオーダーバッファ (隠しレジスタの実現方法)



物理レジスタ

各アーキテクチャレジスタ ↔ 物理アドレスレジスタ

分岐命令ごとにバックアップ表を作り直す必要

リオーダーバッファ方式

命令順に対応付けられている

番号対応表のバックアップは不要

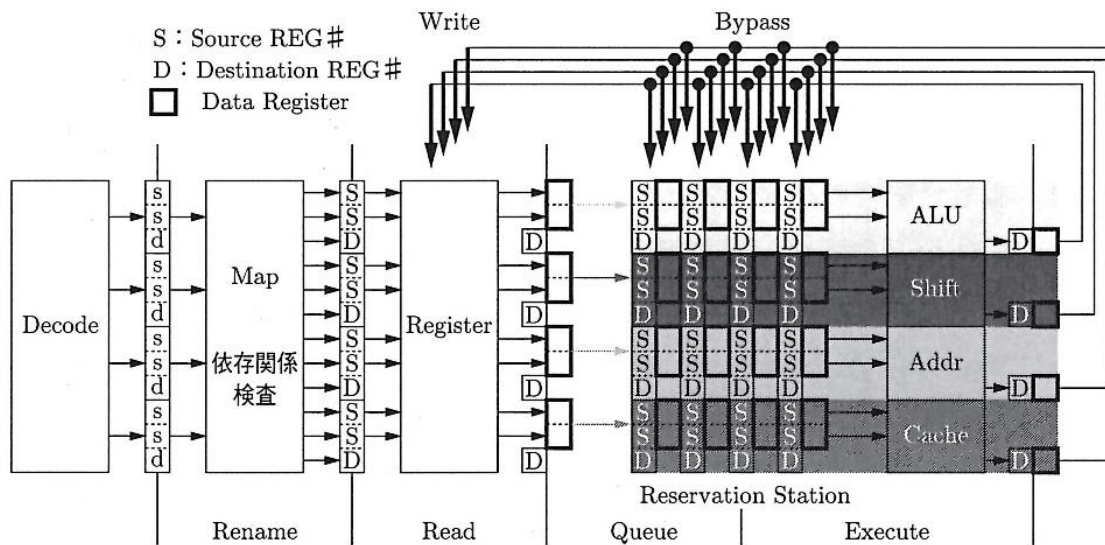
[3] リザベーションステーションと集中命令ウィンドウ

命令順を入れ替える方法
命令の待機方法

Fetch	Decode	Rename	Read	Queue	Execute	Write	Retire			
Fetch	Decode	Rename	Read	Queue		Execute	Write	Retire		
	Fetch	Decode	Rename	Read	Queue		Execute	Write	Retire	
	Fetch	Decode	Rename	Read	Queue	Execute	Write		Retire	
		Fetch	Decode	Rename	Read	Queue	Execute	Write	Retire	
		Fetch	Decode	Rename	Read	Queue		Execute	Write	Retire

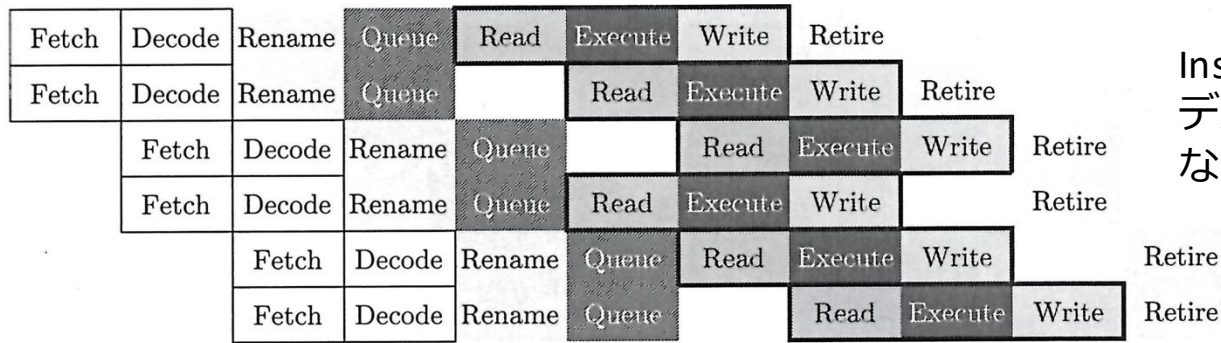
DecodeされたMapステージでレジスタリネーミングが行われる。

演算器毎に決まっているパイプラインに順に送られる。
計算が終わったDestinationの情報も入れられる。

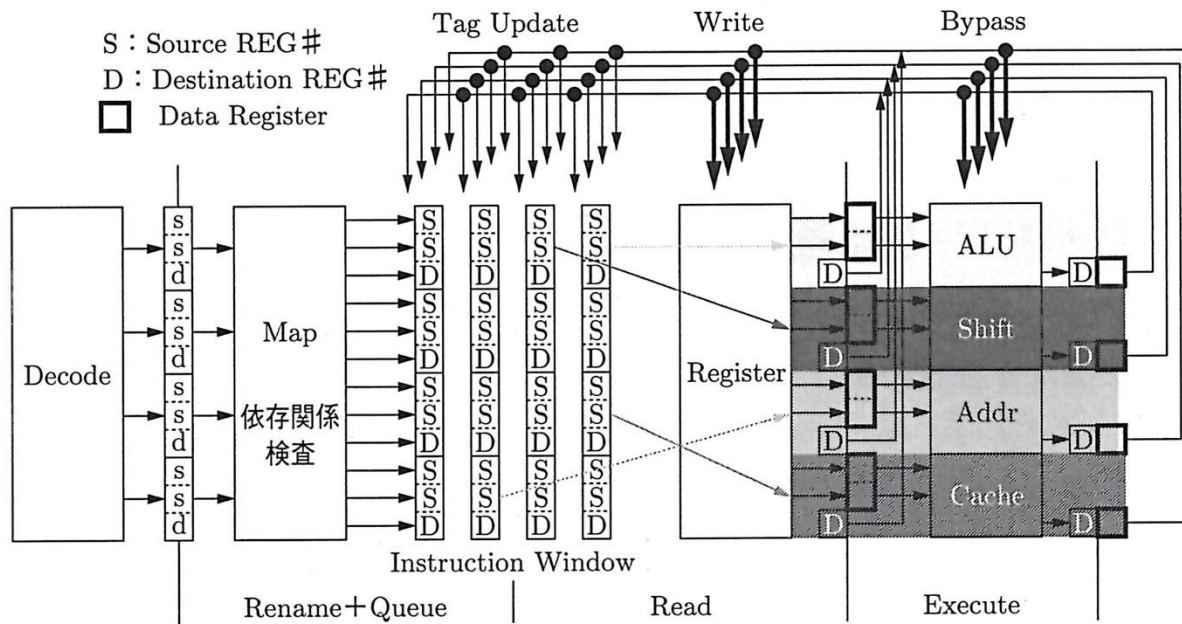


ソースオペランドがそろっていないければ
ここで待機する。

DecodeされたMapステージでレジスタリネーミングが行われる。
(リザベーションステーションと同じ)



Instruction Windowsでソースオペランド、
デスティネーションオペランドの待ちと
なる。



空いているものから送られる。

8.4 ハードウェアによる予測と投機

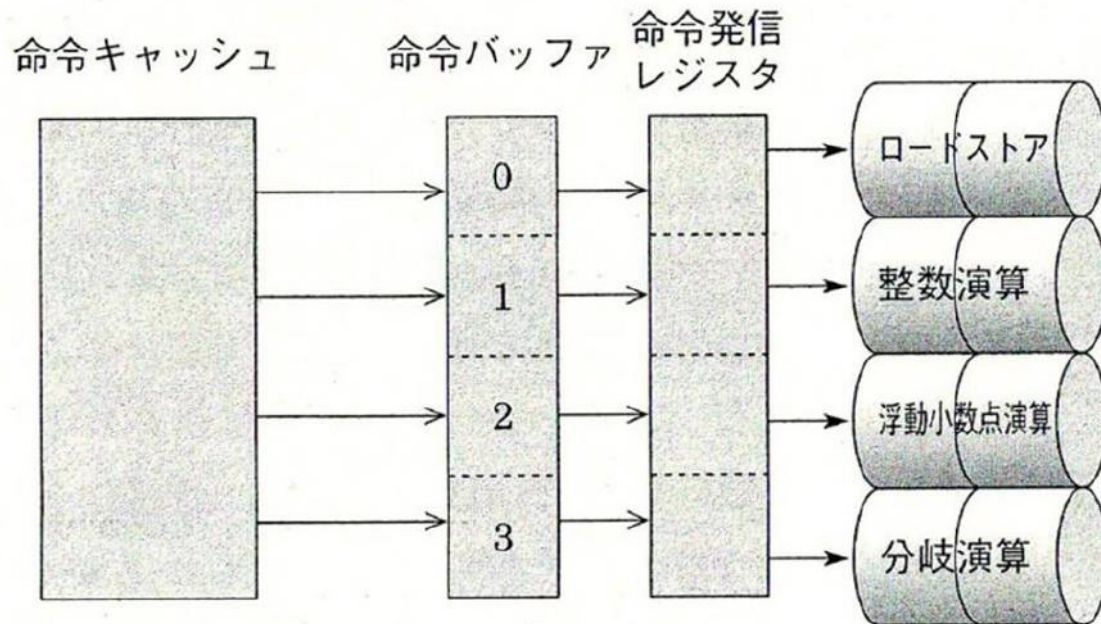
スーパースカラ方式における，将来参照

(分岐予測だけでなく，主記憶アドレスやロード値に関する予測手法)

(アドレス予測や値予測)

last value	最後に使用したアドレスやロード結果をそのまま使う方法
stride based	直近に使用したアドレスやロード結果の差分を外挿する方法
context based	分岐予測におけるGHRと同様，当該ロード命令に至るプログラムの挙動により予測を切り替える方法
last value	以上の複数機構を同時に装備して使い分ける方法

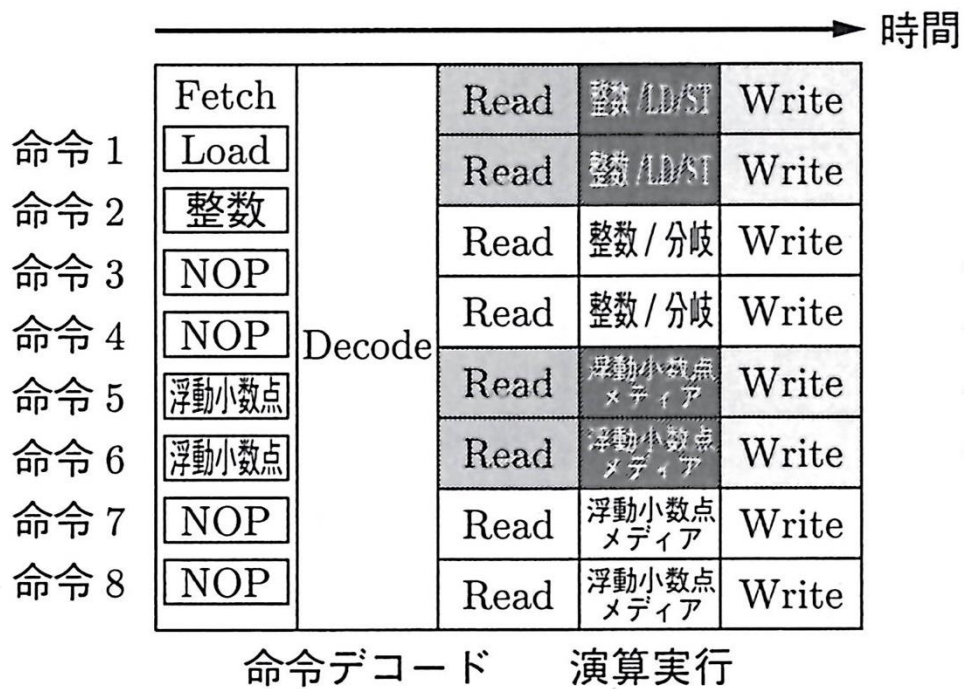
8.5 VLIWとコンパイラ



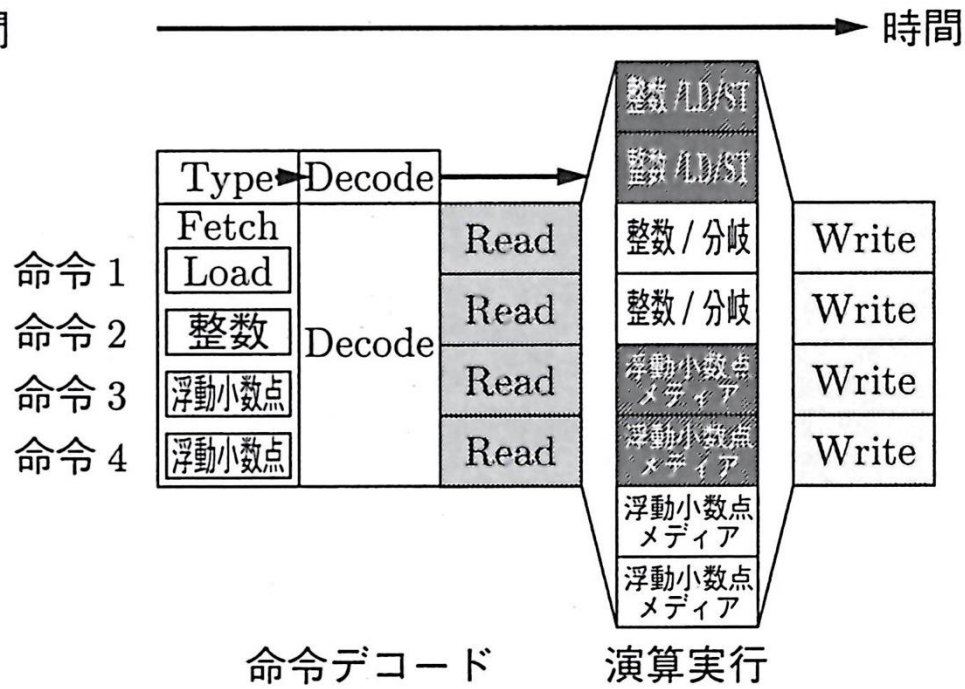
スーパスカラはハードウェアによる動的スケジューリング

VLIW(Very Long Instruction Word) は
命令セットや実行方法を大幅に変更し、コンパイラによる静的スケジューリングと合わせた方法

ハードウェアの負担は少ない



(a) 命令幅 = 演算器数の VLIW 構成



(b) 命令幅 < 演算器数の VLIW 構成

8.6 VLIWの命令スケジューリング

コンパイラの働きが重要

[1] ソフトウェアパイプライニングとアンローリング ソフトウェアパイプライニング

(a) 元の命令列			(b) 並べ換えて実行時間を短縮		
1.	load	(r1+4),r8	1.	load	(r1+4),r8
2.		—1 τ 待ち—	2.	load	(r1+(r5 $\ll$ 2)),r11
3.	add	r8,8,r10	3.	add	r8,8,r10
4.	store	(r1+4),r10	4.	subtract	r10,r11,r12
5.	load	(r1+(r5 $\ll$ 2)),r11	5.	store	(r1+4),r10
6.		—1 τ 待ち—	6.	store	(r1+(r5 $\ll$ 2)),r12
7.	subtract	r10,r11,r12			
8.	store	(r1+(r5 $\ll$ 2)),r12			

(a)はload+浮動小数点加算+storeを30回

(b)は回転数を1/3にし, ソフトウェアパイプラインング

(a) 元の命令列	(b) アンローリング後
1. loop: load (r1+(r5<<2)),f8	1. loop: load (r1+(r5<<2)),f8
2. —1τ 待ち—	2. load (r2+(r5<<2)),f11
3. float-add f8,f9,f10	3. load (r3+(r5<<2)),f14
4. —1τ 待ち—	4. float-add f8,f9,f10
5. —1τ 待ち—	5. float-add f11,f9,f13
6. store (r1+(r5<<2)),f10	6. float-add f14,f9,f16
7. add r5,1,r5	7. store (r1+(r5<<2)),f10
8. compare r5,30	8. store (r2+(r5<<2)),f13
9. branch on less than → loop	9. store (r3+(r5<<2)),f16
	10. add r5,3,r5
	11. compare r5,30
	12. branch on less than → loop

[2] トレーススケジューリングとパーコレーションスケジューリング (広域最適化)

基本ブロック

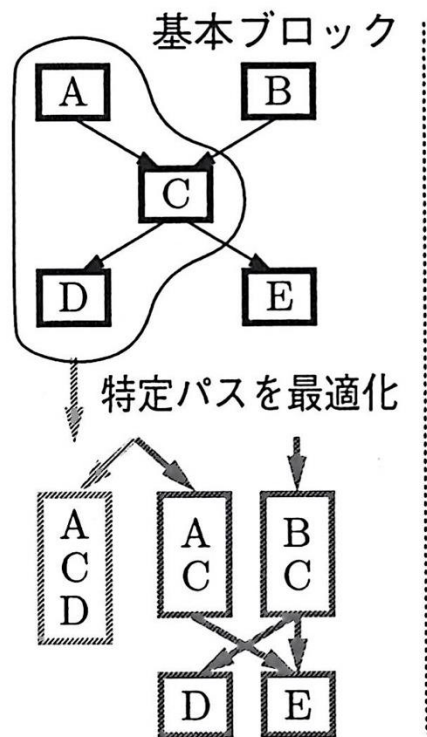
分岐命令による飛び出し/飛込みを境界とした単位をいう

VLIWでは基本ブロックを超えて命令をかき集める必要

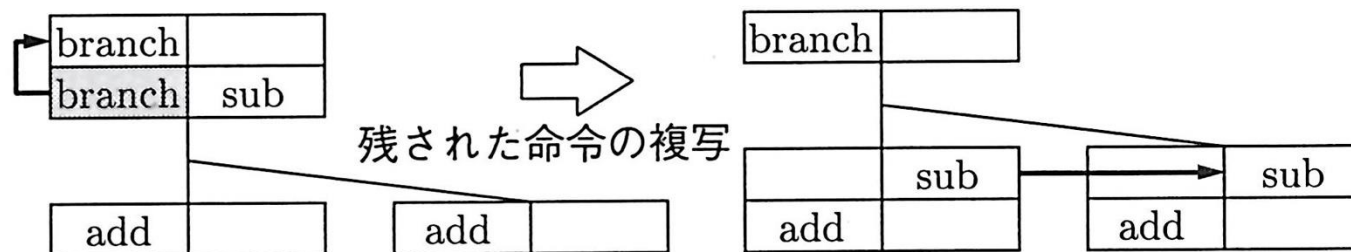
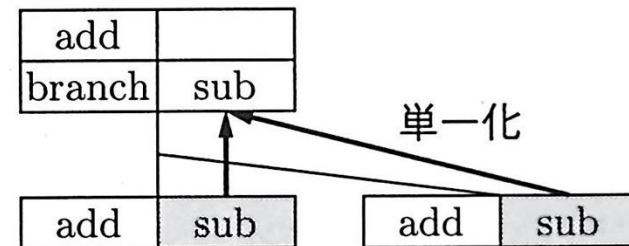
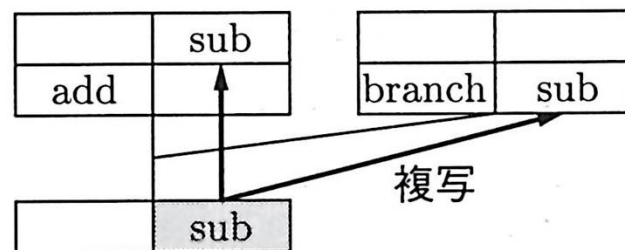
分岐方向によって命令が変化しても無駄にならないようにする必要



巧妙なスケジューリングが必要



(a) トレーススケジューリング



(b) パーコレーションスケジューリング

8.7 コンパイラによる投機と ハードウェアによる支援

(a) ソース プログラム	(b) 通常の命令列	(c) 通常命令の投機	(d) 投機用命令の利用
if (cond) *x = *a + *b;		load a,f1 load b,f2 float-add f1,f2,f3	speculative load a,f1 speculative load b,f2 speculative float-add f1,f2,f3
	branch on !cond	branch on !cond	branch on !cond
	load a,f1 load b,f2 float-add f1,f2,f3 store x,f3	store x,f3	checking store x,f3
		※ !cond は cond の反転を意味する	